

1. Sketch of the solution:

- Idea: Put the first t coins into a min-heap.
- For each c of the remaining coins:
 - If the value of c is bigger than the min of the heap, remove the min and add c .
- Return all the coins in the heap.

2. Task Scheduling

- (a) ii. is optimal as written. Namely, the proof of ii. is just the same as in lecture but mirrored (flip start and finish times). To show that i. is also optimal requires some more care. Here, the events are ordered by finish time. When the next event can be placed into an active channel, it is necessary to pick the channel with the latest finish time where this new event fits. With this extra detail, an exchange argument can be used to prove this greedy rule is optimal (similar to the one used for activity selection). Note that without this extra care, the greedy rule could result in a suboptimal number of channels, eg, [0,1], [0,3], [4,5], [2,6]. Without the extra care, greedy could assign [0,1] and [4,5] to channel 1, but then [0,3] and [2,6] do not fit into channel 2 whereas with the extra care [4,5] would be placed into channel 2 with [0,3], and then [2,6] would fit into channel 1 with [0,1].

- (b) Input: array A of n start-finish time pairs: A[i].s, A[i].f

```
greedy-task-scheduling(A) {
    sort A by start times so that A[1].s <= A[2].s <= ... <= A[n].s
    Create an array channel of length n; // at the end, channel[i] = index of channel for task i.
    Create list F of the final finish time of each active channel.
    for (i = 1 to n) do { // classify the ith request
        addedToSchedule = false // track if A[i] has been added to a channel
        for (j = 1 to length(F)) do {
            if [A[i].s >= F[j]] {\\add A[i] to channel j.
                F[j] = A[i].f; channel[i] = j; addedToSchedule = true
                break
            } }
        if not addedToSchedule{ // create a new channel
            F.add(A[i].f); channel[i] = length(A);
        } }
    return A, channel //need to return reordered A
}
```

Runtime:

- before the loop we do $O(n)$ work to setup,
- the outerloop runs n iterations, each iteration involves some constant work together with an inner loop that considers all active channels (at most n iterations) and does constant work per iteration. Thus, we have $O(n^2)$ time in total.
- Note that the implementation can be improved to $O(n \log n)$ by (similarly to 1. above) using a min-heap to track the finish times F of the active channels (keyed by these finish times). This change would replace the inner loop by checking the min-heap if the minimum value is compatible with the start time A[i].s (if so, we increase the key of this minimum element; otherwise, we add a new channel).

3. See https://en.wikipedia.org/wiki/Prim%27s_algorithm

- **k -Partial Solution G :** For the leftmost k paintings, a set G of guards that covers all of them.
- **Greedy Rule:** For a given k -partial Solution G , let x be the position of the left-most unguarded painting (painting $k + 1$ from the left). Place the next guard g at the position $x + 1$. The idea here is that we need to cover this painting at position x , and we can place g to do so while covering as much as possible further to the right.
- **Correctness:**
 - Let G be a solution constructed by the greedy rule, and let OPT be an optimal solution that shares as much in common with G as possible.
 - If G and OPT are the same, G is optimal and we are done.
 - Suppose G and OPT are different, and let g be the leftmost guard in G that is not in OPT . Further let x be the position of the leftmost painting which caused us to add g into G . Since G and OPT agree on the guards further to the left, none of these guards cover position x . So OPT must have a different guard g^* that is used for the painting at position x . However, since we choose g to be the rightmost guard that can cover the painting at position x , g^* must be on to the left of g . Now consider the new guard set OPT^* obtained from OPT by removing g^* and inserting g . From our choice of g we have that OPT^* still covers all the paintings and uses the same number of guards as OPT , but now OPT^* has more guards in common with G than OPT . This contradicts our choice of OPT , and shows that indeed G and OPT must be the same.
- **Implementation and runtime.**
 - Sort the paintings in increasing order of x_i . Thus we have $x_1 \leq x_2 \leq \dots \leq x_n$.
 - Place the first guard at position $g = x_1 + 1$, and add it to our initial guard set G . (g will always be the rightmost guard in our guard set)
 - For $i = 1$ to n
 - * If $x_i \leq g + 1$ this painting is already guarded, no new guards needed.
 - * Else create a new guard at position $x_i + 1$ and revise g to be $x_i + 1$.
 - **Runtime:** Sorting is $O(n \log n)$, then we loop over all the n paintings doing $O(1)$ work per painting. Thus, $O(n \log n)$ in total.

Solution to Exercise 16.1-3

- For the approach of selecting the activity of least duration from those that are compatible with previously selected activities:

i	1	2	3
s_i	0	2	3
f_i	3	4	6
duration	3	2	3

This approach selects just $\{a_2\}$, but the optimal solution selects $\{a_1, a_3\}$.

- For the approach of always selecting the compatible activity that overlaps the fewest other remaining activities:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	0	1	1	1	2	3	4	5	5	5	6
f_i	2	3	3	3	4	5	6	7	7	7	8
# of overlapping activities	3	4	4	4	4	2	4	4	4	4	3

This approach first selects a_6 , and after that choice it can select only two other activities (one of a_1, a_2, a_3, a_4 and one of a_8, a_9, a_{10}, a_{11}). An optimal solution is $\{a_1, a_5, a_7, a_{11}\}$.

- For the approach of always selecting the compatible remaining activity with the earliest start time, just add one more activity with the interval $[0, 14)$ to the example in Section 16.1. It will be the first activity selected, and no other activities are compatible with it.

[CLRS 23-4] (partial solutions) The focus here is on optimality. We provide an outline of the steps needed to prove this, but the full proofs are not included. Also, efficient implementations are not included.

- This is basically Prim in reverse. In particular, when Prim stops, a set E of (smallest weight) edges will have been seen (a subset of those were chosen for the MST T built by Prim). Importantly, since E includes T , the subgraph $(V(G), E)$ is connected. Thus, the edges beyond E are discarded by this algorithm.

Let T be the output of the given algorithm, and let T^* be an MST that has as many edges in common with T as possible.

If T and T^* are the same, we are done.

So suppose for a contradiction that they are different. Let e be the first edge considered by the algorithm where T and T^* disagree. This gives two cases (1) e is in T but not in T^* ; (2) e is in T^* but not in T . In both cases you need to find a contradiction (doing so is very similar to what we did for Prim's algorithm).

- This is not optimal. Graph G with vertices a, b, c , edges ab, ac, bc , weights $w(ab) = 10, w(ac) = 5, w(bc) = 1$. If the edges are considered as ab, ac, bc , then it will be ab, ac with total cost 15. However, the MST here is made by the edges ac, bc and has cost 6.
- This is optimal, but it follows the approach in Kruskal's algorithm (see section CLRS 23.2) instead of Prim's. To prove it is correct, you should show that at every iteration, T is a set of minimum spanning tree for the **connected components of edges seen up to this point**. You can prove this by induction.